数据库系统
笔记

计科210X 甘晴void 202108010XXX

教材：《数据库系统概论》第6版

数据库系统
笔记
第一篇 基础篇
1 绪论
1.1数据库系统概述
1.2数据模型
1.3数据库系统的结构(三级模式结构)
1.4数据库系统的组成
1.5总结
2 关系数据库
2.1关系
2.1.1关系
2.1.2关系模式
2.2关系操作
2.2.1基本的关系操作
2.2.2关系数据语言的分类
2.3关系完整性
★2.4关系代数
2.4.1传统的集合运算
2.4.2专门的关系运算
3 关系数据库标准语言 SQL
3.1 SQL概述
3.2数据定义
★3.3数据查询
3.4数据更新
3.5空值的处理
3.6视图
4 数据库安全性
自主存取控制
强制存取控制
视图机制
审计
数据加密（略）

甘晴void个性学习笔记

其他安全性保护
5 数据库完整性
5.2 实体完整性
5.3 参照完整性
5.4 用户定义的完整性
5.5 完整性约束命名子句
5.7 触发器
【期中考试备考】
第二篇 设计与应用开发篇
★6 关系数据理论
6.2 规范化
★6.3 数据依赖的公理系统
7 数据库设计
★7.1 数据库设计概述
7.2 需求分析
7.3 概念结构设计
7.4 逻辑结构设计
7.5 物理结构设计
7.6 数据库的实施和维护
【不考】8 数据库编程
8.1 扩展SQL功能
8.2 过程化SQL
8.3 JDBC编程
【不考】9 关系数据库存储管理
10 关系查询处理和查询优化
10.1 关系数据库管理系统的查询处理
10.2 关系数据库管理系统的查询优化
10.3 代数优化
10.4 物理优化
★本章重要简答题
11 数据库恢复技术
11.1 事务的基本概念
★11.3 故障的种类
11.4 数据库恢复的实现技术
★11.5 恢复策略
★11.6 具有检查点的恢复技术
11.7 数据库镜像（了解）
12 并发控制
12.1 并发控制概述
12.2 事务的隔离级别
12.3 封锁
12.4 封锁协议
12.5 活锁和死锁

12.6 并发调度的可串行性
12.7 两段锁协议
12.8 封锁的粒度
【期末考试复习】

HUST电信Void小组学习使用

第一篇 基础篇

1 绪论

1.1数据库系统概述

基本概念：

- 数据：描述事物的符号记录称为数据。数据的含义称为数据的语义，数据与其语义是不可分的。
- 数据库（DB）：长期存储在计算机内、有组织的、可共享的大量数据的集合。数据库中的数据按一定的数据模型组织、描述和存储，具有较小的冗余度、较高的数据独立性和易扩展性，并可为各种用户共享。
- 数据库管理系统（DBMS）：数据库管理软件，计算机的基础软件

  数据定义功能
  数据组织、存储和管理
  数据操纵功能
  数据库的事务管理和运行管理
  数据库的建立和维护功能
  其他功能
- 数据库系统（DBS）：由DB,DBMS及其应用开发工具，应用系统和DBA组成的存储、管理、处理和维护数据的系统
- DBA数据库管理员

数据库管理技术的产生和发展：

人工管理阶段 -> 文件系统阶段 -> 数据库系统阶段

数据库系统阶段的4个优点

- 整体数据的结构化(与文件系统的本质区别)
- 数据的共享性强、冗余度低且易于扩充
- 数据的独立性强（物理独立性，逻辑独立性）
- 数据由数据库管理系统统一管理和控制

另外还需要提供的功能：

- 数据的安全性保护：防止不合法使用造成的数据泄密和破坏。
- 数据的完整性检查：控制数据的正确性，有效性和相容性。
- 并发控制
- 数据库恢复

1.2数据模型

数据模型：数据库系统的核心和基础。

数据建模：把现实世界中的具体事物抽象、组织为某一数据库管理系统支持的数据模型。

概念模型（现实世界抽象为信息世界） -> 数据模型（信息世界转换为机器世界）

概念模型：

- 实体：客观存在并可相互区别的实物称为实体。
- 属性：实体所具有的某一特性。
- 码(key)：唯一标识实体的属性集称为码。
- 实体型：用实体名和属性名集合来抽象和刻画同类实体。例如：学生(学号，姓名，性别，院系)。
- 实体集：同一类型实体的集合称为实体集。
- 联系：实体之间的联系通常是指不同实体集之间的联系。有一对一，一对多和多对多等多种类型。

概念模型的表示方法：实体-联系模型（E-R模型）

数据模型三要素：

- 数据结构：描述数据库的组成对象以及对象之间的联系。
- 数据操作：对数据库中各种对象的实例允许执行的操作的集合，包括操作及有关的操作规则。
- 完整性约束：一组完整性规则。

层次模型：

- 数据库中满足下面两个条件的基本层次联系的集合为层次模型：
    - 有且只有一个结点没有双亲结点，称为根结点。
    - 根以外的其他结点只有一个双亲结点。
- 【注意：层次模型是一种一对多的联系】
- 数据操纵主要为查询，插入，删除，更新。操作要满足层次模型的完整性约束条件。

- 层次模型的优点：
    - 数据结构简单清晰。
    - 查询效率高(因为联系用有向边表示)。
    - 提供了良好的完整性支持。
- 【未标着重】层次模型的缺点：
    - 现实中很多联系是非层次性的。
    - 一个结点有多个双亲结点，就只能引入冗余数据，应用程序编写复杂。
    - 查询子女结点必须通过双亲结点。
    - 结构严密，层次命令趋于程序化。

网状模型：

- 数据库中满足下面两个条件的基本层次联系的集合为网状模型：
    - 允许一个以上的结点无双亲。
    - 一个结点可以有多于一个的双亲。
- 【注意：层次模型是一种多对多的联系】
- 具体的网状数据库系统对数据操纵都加了一些限制，提供了一定的完整性约束。
- 网状模型的优点：
    - 能够更为直接的描述现实世界。
    - 具有良好的性能，存储效率较高。
- 【未标着重】网状模型的缺点：
    - 结构比较复杂，不利于最终用户掌握。
    - DDL，DML复杂，并且要嵌入一种高级语言中，不容易掌握和使用。
    - 记录之间的联系通过存取路径实现，访问数据时必须选择适当的存取路径，用户必须了解系统结构的细节。

关系模型：

- 关系模型要求关系必须是规范化的，关系的每一个分量必须是一个不可分的数据项。
- 关系模型的优点：
    - 建立在严格的数学概念基础上。
    - 概念单一，
    - 关系模型存取路径对用户隐蔽：有更高的数据独立性，更好的安全保密性。（★因此查询需优化）

1.3数据库系统的结构(三级模式结构)

- 模式也称逻辑模式，是数据库中全体数据的逻辑结构和特征的描述。数据库管理系统提供模式数据定义语言(DDL)来严格定义模式。
- 模式反映的是数据结构及其联系，而实例反映的是数据库某一时刻的状态。
- 三级模式结构：
    - 模式：是所有用户的公共数据视图。
    - 外模式：数据库用户能够看见和使用的局部数据的逻辑结构和特征的描述，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。
    - 内模式：数据物理结构和存储方式的描述，是数据在数据库内部的组织方式。
- 为了实现以上三个抽象层次的联系和转换，数据库管理系统在三级模式之间提供了两层映像：外模式/模式映像和模式/内模式映像。
    - 外模式/模式映像：定义了外模式与模式之间的对应关系。
    - 模式/内模式映像：定义了数据全局逻辑结构和存储结构之间的关系。
- 数据库模式即全局逻辑结构是数据库的中心与关键，独立于数据库的其他层次。（设计数据库模式结构时应首先确定）
- 当模式改变时，修改映像使外模式不变，这保证了数据与程序的逻辑独立性。
- 当存储结构(内模式)改变时，修改映像使模式保持不变，这保证了数据与程序的物理独立性。

1.4数据库系统的组成

- 硬件平台及数据库
- 软件
- 人员

人员包括

- 数据库管理员：5个职责（书P32）
- 系统分析员和数据库设计人员
- 应用程序员：编程
- 最终用户

1.5总结

4个基本概念

数据模型3要素

三级模式与两级映射保证逻辑独立性与物理独立性

数据库系统三级模式结构的优点

为什么具有数据和结构的独立性

2 关系数据库

2.1关系

2.1.1关系

域：域

笛卡尔积：笛卡尔积（域的运算）、元组、n元组、分量（元组中的每一个值）、基数 （一个域允许的不同取值个数）

关系：

- 关系（一些域的笛卡尔积的子集即为这些域的关系，抽取有意义的子集），记作R(……)、
- 关系的目/度n（运算的笛卡尔积的个数，即关系表的特征列个数）、
- 属性（n目关系必有n个属性）、

三类关系

基本关系（基本表/基表）【实际存在】有6条性质（列同质、不同元组码不能相同、不同列可出自同一域、列可换顺序、行可换顺序、分量取原子值，即不能表中有表）

查询表【针对查询结果的临时表，实际存在】

视图表【虚表】

2.1.2关系模式

R(A1,A2,A3,)

- 候选码、主码、全码、主属性、非主属性（非码属性）

关系数据库：关系数据库模式：关系数据库的“型”

关系模型的存储结构：RDBMS

2.2关系操作

2.2.1基本的关系操作

- 查询操作
- 数据更新操作

2.2.2关系数据语言的分类

- 关系代数语言
- 关系演算语言（不要求）
- 结构化查询语言（Structured Query Language, SQL）

关系完备性：一个关系数据语言能够表示关系代数可以表示的查询

2.3关系完整性

包括

- 实体完整性（主属性不取空值不得重复）
- 参照完整性（注意外码的定义：不是参照关系的码但是被参照关系的主码）（外码可以取空值或被参考关系中已经存在的值）
- 用户定义的完整性（用户可以自己定义：唯一约束、主键约束、自增长约束、默认值约束、检查约束）

★2.4关系代数

2.4.1传统的集合运算

并、差、交、笛卡尔积

常考：R∩S=R-(R-S)

2.4.2专门的关系运算

基础记号：

- 关系R、t∈R（t是关系R的一个元组）、t[Ai]（元组t中相应于属性Ai的一个分量）
- t[A]=（t[Ai1]，t[Ai2]，t[Ai3]，……，t[Aik]）（元组t中相应于属性Ai的一个分量）
- 元组的连接/串接（就是把列连起来）
- 象集（关系R(X,Z)，对所有列ABCD……,以A=a，B=b（即X=x）这样的条件筛选满足的元组，筛选结果中非列AB的列属性构成的组合即为x在Z上的象集）

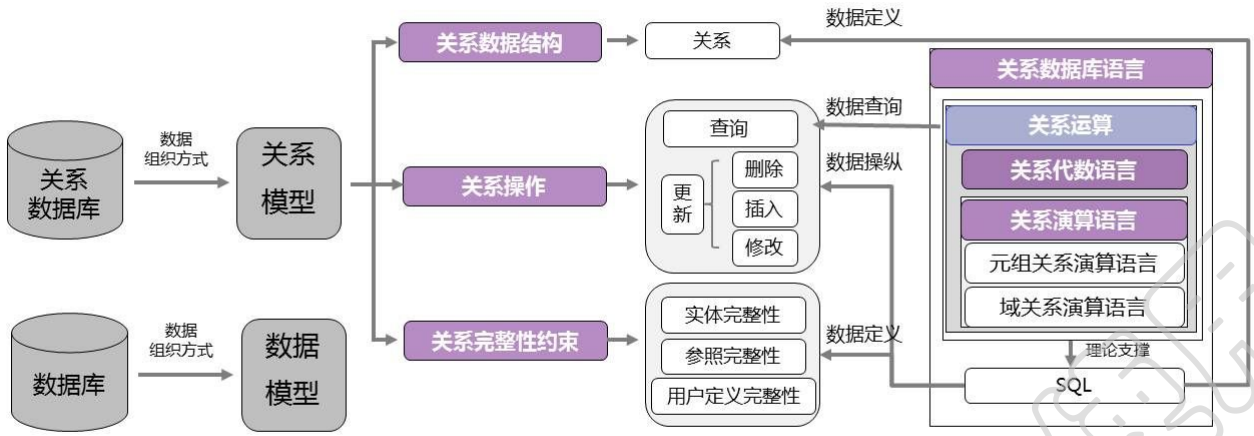
专门关系：

- 选择（σ）：（根据逻辑表达式来筛选有效元组）【注意比较运算符的不等于是<>】
- 投影（Π）：（只关注属性列，筛选出某些属性列并去除重复的元组）
- 连接（⋈）：对两个关系R、S，从笛卡尔积中抽取满足条件的元组（特殊的连接：θ取“=”为等值连接、自然连接）
- 等值连接：抽取R中A列与S中B列相等的元组
- 自然连接：进行比较的列必需是同名的属性列，可能造成悬浮问题
- 悬浮元组、外连接、左外连接（只保留左边关系的悬浮元组）、右外连接（只保留右边关系的悬浮元组）
- 除运算（÷）：（除集合）对于公共属性列的投影，（被除集合）中谁的（象集）完全包含它，谁就需要被纳入结果。
- 【÷运算可以用来做筛选：例如由SC表÷Course表可得到选修所有课程学生的名单】

经过有限次复合得到的表达式称为 关系代数表达式

如何使用五种基本运算来表达专门关系？是一个重要的问题

本章知识框架总结



3 关系数据库标准语言 SQL

3.1 SQL概述

▲特点

- 功能综合且风格统一
- 数据操纵高度非过程化（只要提出做什么就可以）
- 面向集合的操作方式
- 以统一的语法结构提供多种使用方式（独立语言/嵌入式语言）
- 语言简洁且易学易用

3.2数据定义

模式的定义与删除

```
#定义模式
create schema [模式名] authorization <用户名>;
#删除模式
drop schema <模式名><cascade|restrict>                    //级联与限制必须二选一（CASADE会删除所有，而若该模式内有东西，RESTRICT会拒绝删除）
```

基本表的定义、删除与修改

```
#基本表的定义
create table <表名>(<列名><数据类型>[列级完整性约束]
                    ……
                    [,<表级完整性约束>]);

#一些表级完整性约束
PRIMARY KEY(Sno,Cno);
FOREIGN KEY(Sno) REFERENCES Student(Sno);

列级完整性/表级完整性
#基本表的修改
ALTER TABLE <表名>
    [ADD [COLUMN] <新列名><数据类型>[完整性约束]]
    [ADD <表级完整性约束>]
    [DROP [column] <列名> [cascade|restrict]]
    [DROP constraint <完整性约束名> [restrict|cascade]]
    [RENAME COLUMN <列名> to <新列名>]
    [ALTER COLUMN <列名> TYPE <数据类型>]

#基本表的删除
DROP TABLE <表名> [restrict|cascade]  #▲注意区别，默认RESTRICT
```

模式与基本表

```
#基本表属于某一个特定的模式，下面是创建特定模式下基本表的方法。

#可以在指定模式下创建基本表
CREATE TABLE "SC-C-SC".Student（……）

#可以设定模式然后创建基本表
SHOW SEARCH_PATH;#可以查看当前搜索路径
SET SEARCH_PATH TO "SC-C-SC",PUBLIC;#设置当前模式
#之后再创建基本表
```

课本示例

```
#课本示例
/*建立一个学生表*/
CREATE TABLE Student(
    Sno CHAR(9)PRIMARY KEY,
    Sname CHAR(20)UNIQUE,
```

```
    Ssex CHAR(2),
    Sage SMALLINT,
    Sdept CHAR(20)
);
/*建立一个课程表*/
CREATE TABLE Course(
    Cno CHAR(4) PRIMARY KEY,
    Cname CHAR(40) NOT NULL,
    Cpno CHAR(4),
    Ccredit SMALLINT,
    /*表级完整性约束条件*/
    FOREIGN KEY (Cpno) REFERENCES Course(Cno)
);
/*建立学生选课表SC*/
CREATE TABLE SC(
    Sno CHAR(9),
    Cno CHAR(4),
    Grade SMALLINT,
    PRIMARY KEY(Sno,Cno),
    FOREIGN KEY(Sno) REFERENCES Student(Sno),
    FOREIGN KEY(Cno) REFERENCES Cource(Cno)
);
```

数据类型

P75

索引的建立与删除

建立索引：加快查询速度

索引是关系数据库管理系统的内部实现技术，属于内模式的范畴。

```
#索引的建立
CREATE [unique][cluster]index <索引名>
on <表名>(<列名>[<次序>],……)
#索引的修改
ALTER index <旧索引名> rename to <新索引名>
#索引的删除
DROP index <索引名>

#e.g.
CREATE UNIQUE INDEX Stusno ON Student(Sno);
CREATE UNIQUE INDEX Cousno ON Cource(Cno);
CREATE UNIQUE INDEX SCno ON SC(Sno ASC,Cno DESC);

```

数据字典（记录数据定义）

★3.3数据查询

```
SELECT [all|distinct]<目标列表达式>[别名][,<目标列表达式>[别名]]……
FROM <>
[WHERE<条件表达式>]
[GROUP BY <某个列名> [having <条件表达式>]]
[ORDER BY <某个列名> [asc|desc]]
[limit <行数1>[offset <行数2>]]
```

3.3.1单表查询

```
#选择表中的若干列（投影操作）
SELECT Sno FROM Student;

#选择表中的若干组（选择操作）
SELECT DISTINCT Sno FROM Student;
#比较大小
WHERE = > < <>  != !> !<
#确定范围
SELECT Sno FROM Student WHERE Sage BETWEEN 20 AND 21;
SELECT Sno FROM Student WHERE Sage NOT BETWEEN 20 AND 21;
#确定集合
IN | NOT IN
#字符匹配

```

```
% #任意长度字符串
_ #任意单个字符
WHERE Sname LIKE '__寅';
WHERE Sname NOT LIKE '%寅';
#涉及到含有通配符，可以后跟ESCAPE'\'进行转义
#WHERE Sname NOT LIKE '%寅\_' ESCAPE '\';
#涉及空值的查询
WHERE GRADE IS NULL
WHERE GRADE IS NOT NULL
#多重条件查询
AND OR 括号

#ORDER BY子句
ORDER BY grade DESC | ASC #（排序操作）允许第二关键字
#查询结果排序，不影响原表

#聚集函数（数理统计）
#不能用作WHERE子句条件表达式
#只能作为SELECT子句和GROUP BY子句的HAVING短语
COUNT(*)
COUNT([ DISTINCT | ALL ]<列名>)
支持 sum avg max min
#e.g.
SELECT COUNT(DISTINCT Sno) From SC;

#GROUP BY子句（分组筛选）★★★
GROUP BY
SELECT Sno FROM SC GROUP BY Sno HAVING COUNT(*) > 10;

#Limit子句（只选择前多少个子句，常与ORDER BY连用）
LIMIT 10 OFFSET 5 #显示10个忽略前5个

```

3.3.2连接查询

```
# 自然连接查询 复合条件连接查询
SELECT student.sno,sname
FROM student,sc
WHERE student.sno=sc.sno AND
    sc.cno='2' AND sc.grade>90;
```

```
# 自身连接，需要为表起别名
#e.g. 间接先修课（先修课的先修课）
SELECT first.cno,seconde.cpno
FROM course first,course second
WHERE first.cpno=second.cno;
# 外连接，把表中的悬浮元组保存在结果关系中
SELECT student.sno,sname,ssex,sage,sdept,cno,grade
FROM student LEFT OUTER JOIN sc ON (student.sno=sc.sno);
# 多表连接和以上类似

```

3.3.3嵌套查询

查询块：一个select-from-where

嵌套查询，外层查询（父查询），内层查询（子查询）

【子查询不能用order by】

any,all,

exist谓词：只返回逻辑真假
存在量词与全称量词

```
  # 不相关子查询
SELECT name
FROM student
WHERE dept IN
    (
        SELECT dept
        FROM student
        WHERE name='刘晨'
    );
# 相关子查询
SELECT sno,cno
FROM SC x
WHERE Grade >= (
                SELECT AVG(Grade)
                FROM SC y
                WHERE y.sno=x.sno
               );
# ANY(SOME)/ALL
```

```sql
SELECT name,age
FROM student
WHERE age<ALL
    (
        SELECT age
        FROM student
        WHERE dept='CS'
    )
    AND dept!='CS';
# EXIST
SELECT name
FROM student
WHERE EXISTS
    (
        SELECT *
        FROM SC
        WHERE sno=student.sno AND cno='1'
    );

```

★全称量词的转换

看书

★逻辑蕴含的转换

看书和作业例题

3.3.4集合查询（了解即可）

```sql
# UNION
SELECT *
FROM student
WHERE dept='CS'
UNION (UNION ALL)
SELECT *
FROM student
WHERE age<=19;

```

```
# INTERSECT
SELECT *
FROM student
WHERE dept='CS'
INTERSECT
SELECT *
FROM student
WHERE age<=19;

# EXCEPT
SELECT *
FROM student
WHERE dept='CS'
EXCEPT
SELECT *
FROM student
WHERE age<=19;

```

3.3.5基于派生表的查询

```
SELECT Sno,Cno
FROM SC,(SELECT Sno,Avg(Grade) FROM SC GROUP BY Sno)
    AS Avg_SC(Avg_sno,Avg_grade)
WHERE SC.Sno = Avg_SC.Avg_sno AND SC.Grade >= Avg_SC.Avg_grade;
```

3.4数据更新

3.4.1插入数据

```
插入一个元组
INSERT INTO <表名> [(<属性列1>,<属性列2>，……)]          #如果与原表一致则可以省略
VALUES (<常量1>,<常量2>，……);

插入子查询结果
insert into <表名> [(<属性列1>,<属性列2>，……)]
子查询;

#e.g.
# 插入元组
```

```
INSERT INTO student(sno,sname,ssex,sdept,sage)                    #可以不指出属性名，但VALUE要保持顺序
VALUES('201215128','陈东','男','IS',18);                    #未给出的列将自动赋空值
# 插入子查询结果
CREATE TABLE DEPT_AGE
    (SDEPT CHAR(15)
     AVG_AGE SMALLINT);
INSERT DEPT_AGE(SDEPT,AVG_AGE)
SELECT SDEPT,AVG(SAGE)
FROM STUDENT
GROUP BY SDEPT;
```

3.4.2修改数据

```
update <表名>
set <列名>=<表达式>,<列名>=<表达式>……
[where <条件>];

# 修改一个元组的值
UPDATE student
SET sage=22
WHERE sno='201215121';
# 修改多个元组的值
UPDATE student
SET sage=sage+1;
# 带子查询的修改语句
UPDATE SC
SET grade=0
WHERE sno IN
    (SELECT sno
     FROM student
     WHERE sdept='CS');

```

3.4.3删除数据

```
delete from <表名>
[where <条件>];
```

3.5空值的处理

```
空值与另一个值的算数运算结果为空值
空值与另一个值的比较运算结果为UNKNOWN，不参与/影响是非判定
```

3.6视图

```
CREATE VIEW <视图名>[(<列名1>,<列名2>，……)]
as <子查询>     #子查询可以嵌套，此时称建立在多个基本表上
[with check option];     #加上该句之后插入值时会自动检查是否符合子查询的要求，若不满足则拒绝
DROP VIEW <视图名> [CASCADE]

UPDATE SET WHERE
INSERT INTO VALUES
DELETE FROM WHERE
```

- 执行CREATE VIEW 语句时指示把视图的定义存入数据字典，不执行SELECT语句，只有在对视图查询时，才按视图的定义从基本表中将数据查出。
- 行列子集视图：从单个基本表导出，并且只是丢掉了某些行或列，但保留了主码。（一般是可更新的）
- 带表达式的视图：带虚拟列的视图。
- 视图消解：首先进行有效性检查，检查查询中涉及的视图等是否存在。若存在，则从数据字典中取出视图的定义，把定义中的子查询和用户的子查询结合起来，转换成等价的对基本表的查询，然后再执行修正了的查询

★视图的作用：

- 视图能够对机密数据提供安全保护
- 视图对重构数据库提供了一定程度的逻辑独立性
- 视图能够简化用户操作
- 视图使用户能以多种角度看待同一数据

4 数据库安全性

数据库安全性是指保护数据库，以防不合法使用所造成的数巨泄露、篡改或破坏。

★数据库的安全性保护数据防止恶意的破坏和非法的存取

不安全因素：

- 非授权用户对数据库的恶意存取和破坏
- 重要或敏感的数据被泄露
- 安全环境的脆弱性

安全标准：TCSEC：A1,B3,B2,B1,C2,C1,D

CC

用户身份鉴别：静态口令鉴别，动态口令鉴别，生物特征鉴别，智能卡鉴别，入侵检测

存取控制：自主存取控制DAC（C2级），强制存取控制MAC（B1级）

自主存取控制

自主存取控制的缺点是可能存在数据无意识的泄露。因为数据本身并无安全性标记。

授权（grant）

```
GRANT <权限1>,<权限2>.....      #全部，用all priviliges
ON <对象类型><对象名>,<对象类型2><对象名2>.....
TO <用户1>,<用户2>.....         #全部，用public
[with grant option]     #是否允许授权传播
#注意不得循环授权
```

收回（revoke）

```
revoke <权限1>,<权限2>.....
on <对象类型><对象名>,<对象类型2><对象名2>.....
from <用户1>,<用户2>.....[cascade | restrict]
```

创建数据库权限

```
create user <username> [with] [superuser | createdb] | password
'password';
```

数据库角色

```
create role <角色名>                    #创建角色
grant <权限> on <对象类型><对象名> to <角色>          #角色授权
grant <角色> to <角色/用户> [with admin option] #角色授予其他角色/用户，with admin option是否允许传播
revoke <权限> on <对象类型><对象名> from <角色>      #角色授权收回
```

强制存取控制

强制存取控制是对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体。

同时检查DAC和MAC

每个数据库对象被标以一定的密级，每一个用户被授予某一个级别的许可证。对于任意一个对象，只有具有合法许可证的用户才可以存取。主体对客体的存取必须遵循以下规则：

- 主体许可证级别大于或等于客体密级时，才能读取客体
- 主体许可证级别小于或等于客体密级时，才能写客体

第二条是为了防止高许可证级别的主体恶意降低高密级客体导致数据泄露。

视图机制

与授权机制结合

审计

审计功能把用户对数据库的所有操作记录下来放入审计日志。

审计员可以通过审计日志监控数据库行为，找到非法存取数据的人，时间和内容等。审计机制提供了一种事后安全检查的机制。

DDL,DML,DCL分别是什么

```
show audit_trail
set audit_trail to on
audit alert,update on xxx by access #对xxx进行审计
noaudit alert,update on xxx      #取消对xxx进行审计
```

数据加密（略）

存储加密，传输加密

其他安全性保护

推理控制，隐蔽信道，数据隐私，“三权分立”的安全管理机制

5 数据库完整性

★数据库的完整性防范对象：不合语义、不正确的数据，防止它们进入数据库。

数据库数据的正确性和相容性

据库管理系统必须能够实现以下功能：

- 提供定义完整性约束的机制
- 提供检查完整性约束的方法
- 提供完整性的违约处理办法

5.2 实体完整性

PRIMARY KEY 主码

检查主码值是否唯一，检查主码的各个属性是否为空，否则拒绝修改

5.3 参照完整性

FOREIGN KEY 外码

对被参照表和参照表进行修改时可能出现问题

违约处理策略：（一般采用默认，如果需要其他，则在创建参照表时显式说明）

- 拒绝执行（默认）
- 级联操作
- 设置为空值

5.4 用户定义的完整性

属性上的约束（不满足拒绝执行）

- 列值非空（NOT NULL）
- 列值唯一（UNIQUE）
- 检查列值是否满足一个条件表达式（CHECK短语）

元组上的约束

违约处理：拒绝执行

注意如何使用CHECK子句

5.5 完整性约束命名子句

完整性约束命名子句

```
CONSTRAINT <完整性约束名> <完整性约束>
```

包括NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK等子句

修改表中的完整性限制

```
ALTER TABLE xxxxx
    DROP CONSTRAINT C1
    ADD CONSTRAINT C2 NOT NULL
```

5.7 触发器

事件-条件-动作规则（ECArule）

类似约束，自动激活，功能更强

定义触发器

```
CREATE TRIGGER <触发器名>
{BEFORE|AFTER} <触发事件> ON <表名>          #触发事件为INSERT,DELETE,UPDATE以及它们的and,or以及类似UPDATE <触发列,……>具体指明操作列
REFERENCING NEW|OLD AS <变量>
FOR EACH {ROW|STATEMENT}
[WHEN<触发条件>]
<触发动作体>
```

教材举例：

```
# 将INSERT语句增加的元组数记录到insertlog中
CREATE TRIGGER Student_Count
AFTER INSERT ON Student
REFERENCING
    NEW TABLE AS DELTA
FOR EACH STATEMENT
    INSERT INTO StudentInsertLog(Numbers)
    SELECT COUNT(*) FROM DELTA;
# 将分数增加10%的操作添加到一个另一个表SC_U
CREATE TRIGGER SC_T
AFTER UPDATE ON SC
REFERENCING
    OLDROW AS OldTuple
    NEWROW AS NewTuple
FOR EACH ROW
WHEN (NewTuple.Grade>=1.1*OldTuple.Grade)
    INSERT INTO SC_U (Sno,Cno,OldGrade,NewGrade)

VALUES(OldTuple.Sno,OldTuple.Cno,ldTuple.Grade,NewTuple.Grade);
# 教授的工资不低于4000，低于4000则改为4000
CREATE TRIGGER Insert_Or_Update_Sal
BEFORE INSERT OR UPDATE ON Teacher
REFERENCING NEW row AS newtuple
FOR EACH ROW
BEGIN
    IF(newtuple.Job='教授') AND newtuple.Sal<4000)
        THEN newtuple.Sal:=4000;
    END IF;
END;

```

执行触发器（顺序）

- BEFORE触发器
- 激活该触发器的SQL语句
- AFTER触发器

删除触发器

```
1  DROP TRIGGER <触发器名> ON <表名>
```

HUST电脑void小组学习笔记

【期中考试备考】

事务的四个特性：持久性，一致性，原子性，隔离性

SQL语言具有的功能：数据定义，数据操纵，数据控制

DML数据操纵语言，DBL数据定义语言

HUT电脑void小组学习使用

第二篇 设计与应用开发篇

★6 关系数据理论

6.2 规范化

函数依赖：

- X函数确定Y　或　Y函数依赖于X　记作　X→Y
- 非平凡的函数依赖/平凡的函数依赖（X→Y但Y⊆X，则是平凡的）
- 完全函数依赖/部分函数依赖（X推出Y时X中的各部分是否缺一不可，若X的一部分就可以推出Y，则是部分函数依赖）
- 直接函数依赖/传递函数依赖（X→Y,Y不是平凡的,Y不能→X,Y→Z,Z不是平凡的，则X→Z有传递函数依赖）

（★不考）多值依赖：

- 对于关系R(U,F),X+Y+Z=U,若存在(x,y1,z1),(x,y2,z2)时，(x,y1,z2),(x,y2,z1)也同时存在，则多值函数依赖X→→Y。
- 理解：x确定时，y1到z的所有映射，y2都需要有。
- 若Z=空集，则X→→Y为平凡的多值依赖

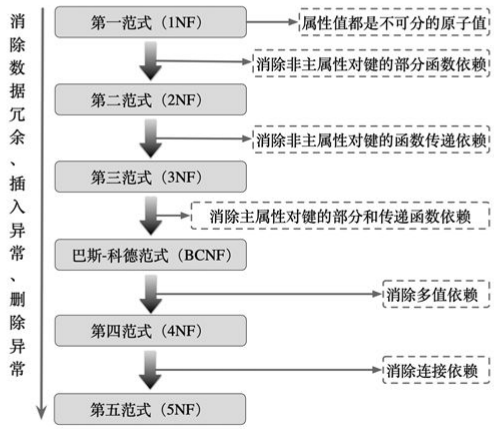
码/键：

- 候选码：K-（F）->U，（U完全函数依赖于K），则K为R的候选码。
- 主码：若候选码多于一个，可以指定一个为主码。
- 全码：主码就是全部属性时。
- 超码：K-（P）->U，（U部分函数依赖于U），则K为超码。
- 外码：R中属性组X不是R的码，但X是另一个关系模式的码，X是R的外码。
- 候选码的超集（若存在）一定是超码，候选码的任何真子集一定不是超码

范式

范式和规范化

➢ 一个低一级范式的关系模式，通过模式分解可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化(normalization)。



规范化过程和各范式示意图

29

★不同等级的范式

- 1NF（属性值都是不可分的原子值）
- 2NF（每一个非主属性完全函数依赖于任何一个候选码）
- 3NF（在1NF基础上，R不存在码X属性组Y非主属性Z且Z不是Y的子集，X→Y，Y→Z，Y不能→X）【每一个非主属性不传递依赖于码，不部分依赖于码】
- BCNF（在1NF基础上，X→Y且Y不是X子集，X必含有码，即每一个决定因素必含有码）【不存在任何属性对候选码的传递函数依赖】在3NF基础上消除了主属性对码的部分函数依赖
- （不考）4NF（在1NF基础上，对于R的每个非平凡多值依赖X→→Y且Y不是X的子集，X都含有码）

（应试）掌握到3NF与BCNF即可，考题问是否能达到3NF

★6.3 数据依赖的公理系统

Armstrong公理系统

- A1 自反律：若Y⊆X⊆U，则F逻辑蕴涵X→Y
- A2 增广律：若F逻辑蕴涵X→Y，Z⊆U，则F逻辑蕴涵XZ→YZ
- A3 传递律：若F逻辑蕴涵X→Y,Y→Z，则F逻辑蕴涵X→Z

根据A1,A2,A3可得以下推理规则

- 合并规则：若X→Y,X→Z，则X→YZ
- 分解规则：若X→Y,Z⊆Y，则X→Z
- 伪传递规则：若X→Y,WY→Z，则WX→Z

引理：X→A1A2A3A4……Ak <<<充分必要>>> X→Ai成立（1<i<n）

Armstrong公理系统的有效性&完备性

- 有效性：由F出发，根据系统推导出来的每一个函数依赖一定在F+中
- 完备性：F+中的每一个函数依赖，必定可以由F触发根据系统推导出来

函数依赖集的闭包：

- F为U上一组函数依赖，X⊆U，X在F上的闭包={A | X→A能由F根据Armstrong公理系统推导出}
- 引理6.2：X,Y⊆U,X→Y能由F根据Armstrong公理系统推导出 <<<充分必要>>> Y⊆X在F上的闭包

函数依赖集的等价

- 引理6.3【F的闭包=G的闭包】<<<充要条件>>>【F的闭包⊆G的闭包】 &&【G的闭包⊆F的闭包】
- 判断是否【F⊆G的闭包】成立：逐一对F中的函数依赖X→Y，判断Y⊆（G+在X中的闭包）是否成立即可。

★★★大题考察：

【书P187-189】

- 求某个属性组X在F上的闭包（【做法】由X和组成X的元素去做扩散，看能推出哪些元素，自己自然是可以推出自己的）
- 判断该属性组X是否为候选码（闭包为全部属性的集合）（【做法】如果推出的元素就是全集，那这就是候选码）
- （书上没有）求解关系的候选码（【思路】分为四类，L,R,LR,N，分别是只在左边出现，只在右边出现，左右都出现，左右都不出现，L和N必定是候选码中的属性，R必定不是，LR可能是）
- 判断两个函数依赖集是否等价（【做法】假设对F和G判定，对F中所有的X→Y都做X在G中的闭包，看看Y是不是被包含，然后同理可得G中也是，就能证明是不是等价的）
- 计算最小依赖集/最小覆盖（【思路】①将右边拆成单个属性②试图将左边拆成单个属性：若AB→C，若C∈A在F中的闭包，则B冗余（剥夺B，不影响，B多余）③试图删除某个关系：若 A→B，若B∈A在{F删去A→B}的闭包，则该关系冗余（剥夺该关系，该关系的左值仍能推出右值，该关系冗余））

- 【具体做法】【图片】
- 检验F是否为最小依赖集（计算出来是自己）

7 数据库设计

★7.1 数据库设计概述

数据库一般设计过程

- 需求分析阶段：准确了解与分析用户需求（包括数据与处理），需求分析如果不好可能会导致整个数据库重做。
- 概念结构设计阶段：通过对用户需求进行综合、归纳与抽象，形成一个独立于具体数据库管理系统的概念模型。
- 逻辑结构设计阶段：将概念结构转换为某个数据库管理系统所支持的数据模型，并对其进行优化。
- 物理结构设计阶段：为逻辑数据模型选取一个最适合应用环境的物理结构（包括存储结构和存取方法）。
- 数据库实施阶段：设计人员运用数据库管理系统提供的数据语言、工具及宿主语言，根据逻辑设计和物理设计的结果建立数据库，编制与调试应用程序，组织数据入库，并进行试运行。
- 数据库运行和维护阶段：在数据库系统运行过程中对其进行评价、调整与修改。

7.2 需求分析

数据字典，包括下列5项

- 数据项：不可再分的数据单位
- 数据结构：反映数据项之间的组合关系。数据结构可由数据项/数据结构混合组成。
- 数据流：数据结构在系统内传输的路径
- 数据存储：数据结构停留或保存的地方，也是数据流的来源和去向之一。
- 处理过程：判定表/判定树

7.3 概念结构设计

概念模型：①真实充分反映现实世界，②易于理解，③易于更改，④易于向关系模型，网状模型，层次模型等数据模型进行转换。

E-R模型：

实体之间的联系：

- 两个实体型之间：一对一联系，一对多联系，多对多联系
- 两个以上实体型：
- 单个实体型内：

E-R图：

- 实体型：矩形
- 属性：椭圆形
- 联系：菱形
- 无向边：需要标注联系的类型(1:1,1:m,m:n等)

【考试：先画含属性的实体型图，再画一张不含属性但含联系的实体型图】

用E-R图进行概念结构设计：

- 实体与属性划分原则：属性为①不可再分的数据项 ②与别的实体无联系

E-R图集成：

- 自顶向下需求分析，自底向上设计概念结构

过程：

（1）合并E-R图，生成初步E-R图

- 属性冲突：属性域冲突，属性取值单位冲突
- 命名冲突：同名异义，异名同义
- 结构冲突：①同一对象在不同应用中具有不同的抽象 ②同一实体在不同子系统的E-R图中包含的属性个数和次序不一致 ③.....

（2）消除不必要的冗余，设计基本E-R图

- 分析法：【.....不是考察重点，不再展开.....】
- 规范化理论：【.....不是考察重点，不再展开.....】

7.4 逻辑结构设计

E-R图向关系模型的转换

- 1:1的联系：可以转换为一个独立的关系模式，也可以与任意一端对于的关系模式合并。
- 1:n的联系：可以转换为一个独立的关系模式，也可以与n端对应的关系模式合并。
- m:n的联系：转换为一个独立的关系模式。各实体的码组成关系的码或关系码的一部分。
- 三个或三个以上实体间的一个多元联系可以转换为一个关系模式
- 具有相同码的关系模式可以合并

数据模型的优化

以规范化理论为指导：【5条】

水平分解：把（基本）关系的元组分为若干子集合

垂直分解：把关系模式R的属性分解为若干子集合

7.5 物理结构设计

- 物理结构：主要是存取方法和存取结构
- 对物理结构进行评价：时间/空间效率

数据库物理结构设计的内容和方法

选择关系模式存取方法

（1）B+树索引方法的选择

- 一个/一组属性经常在查询条件中出现，考虑在这个/这组属性上建立索引
- 一个属性经常作为最大值和最小值等聚集函数的参数，考虑在这个属性上建立索引
- 一个/一组属性经常在连接操作的连接条件中出现，考虑在这个/这组属性上建立索引。

维护索引要代价，索引不是越多越好

（2）哈希索引方法的选择

①大小可预知且不变，或②关系大小动态改变，但DBMS支持动态哈希索引方法

（3）聚簇方法的选择

提高某个属性（组）的查询速度，把这个或这些属性上具有相同值的元组集中存放在连续的物理块。该属性（组）称聚簇码。

设计候选聚簇【3条，略】

检查候选聚簇中的关系，取消不必要的关系【3条，略】

确定数据库的存储结构

- 确定数据存放位置
- 确定系统配置

评价数据库的物理结构：存储空间，存取时间，维护代价等

重组和重构

- 数据库重组：不修改原设计的逻辑和物理结构
- 数据库重构：部分修改数据库的模式和内模式

7.6 数据库的实施和维护

数据的载入和应用程序的编码与调试

数据库的试运行

数据库的运行和维护

- 数据库的转出和恢复
- 数据库的安全性/完整性控制
- 数据库性能的监督，分析和改造
- 数据库的重组与重构：重组（不修改逻辑和物理结构），重构（部分修改数据库的模式和内模式）

Void学习组

【不考】8 数据库编程

8.1 扩展SQL功能

with子句

```
1
```

WITH RECURSIVE 子句

```
1
```

内置函数

```
1
```

通过高级语言实现复杂应用

8.2 过程化SQL

过程化SQL的基本结构

```
1
```

变量/常量定义

```
1
```

流程控制：条件控制/循环控制/错误处理

```
1
```

存储过程：创建/执行/修改/删除/优点

HUE-Void-仅供学习使用

```
1
2
3 # 优点
4 运行效率更高，降低了客户机和服务器间的通信量，方便实施企业规则
```

存储函数（必须指定返回类型）：创建/执行/修改

```
1 CREATE OR REPLACE FUNCTION 函数名
2 RETURNS <类型>
3 AS <过程化SQL块>
```

8.3 JDBC编程

Java DataBase Connectivity

结构：用户应用程序，JDBC驱动程序管理器，数据源

使用JDBC操纵数据库的工作流程

- 加载驱动程序
- 建立于数据库的连接
- 执行SQL语句
- 处理结果集
- 释放资源

【书P263有完整案例】

【不考】9 关系数据库存储管理

了解 B+树索引，哈希索引

10 关系查询处理和查询优化

10.1 关系数据库管理系统的查询处理

查询处理步骤：

- 查询分析（ 词法+语法），
- 查询检查（语义，符号名转换安全性，完整性，这一步生成语法树），
- 查询优化（代数优化：关系代数表达式的优化，物理优化：存取路径和底层操作算法的优化，生成查询执行计划），
- 查询执行（代码生成器生成具体代码）

实现查询操作的算法：

选择操作：

- 全表扫描算法
- 索引扫描算法
- （选择率大小不同，效率不同，根据结果集来选择，如果结果集比较小，就用索引，否则全表扫描）

连接操作：

- 嵌套循环连接算法（假设内存最大M块，A表先读M-1块，B表先读1块，进行匹配，然后B表下一块，B表读完后，A表再继续下一个M-1块，直至全部结束），
- 排序合并连接算法（先对连接属性排序，然后对于同一个A表按顺序读B表，读到B表第一个连接属性不同的就返回A表读下一个），
- 索引连接算法（建立连接属性的索引，然后根据索引查找并连接），
- 哈希连接算法（划分/创建阶段：以连接属性作为哈希码，把较小的表映射到桶中，试探/连接阶段：另一个表进行散列，然后桶中再进行匹配）

10.2 关系数据库管理系统的查询优化

查询优化：选择有效的策略，求得给定关系表达式的值，使得查询代价较小。

★一个实例P308【可考计算题】

10.3 代数优化

通过对关系代数表达式的等价变换来提高查询效率

关系代数表达式等价变换规则【书P309-310】

语法树的启发式优化【5条启发式规则】

★大题：画语法树，关系代数语法树，优化后的语法树

10.4 物理优化

可供选择的方法有

- 基于启发式规则的优化
- 基于代价估算的优化
- 两者结合

基于启发式规则的优化：

选择操作的启发式规则：

- 小关系：用全表顺序扫描
- 大关系：
- 【5条】书P313

连接操作的启发式规则：

- 【4条】书P313
- 选用嵌套循环连接算法时，选择其中占用块数B较小的表作为外表

基于代价估算的优化

【书P314】

★本章重要简答题

1.试述关系数据库管理系统查询优化的一般准则。

记忆：选择怎么样？投影怎么样？选择和投影可以怎么样？选择和笛卡尔积可以怎么样？公共子表达式？连接算法？

```
（1）尽可能先做选择运算
（2）投影运算和选择运算应同时进行
（3）把投影和其前后的双目运算结合起来
（4）把某些选择与在它前面要执行的笛卡尔积结合起来成为一个连接运算
（5）找出公共子表达式
（6）选取合适的连接算法
```

2.试述关系数据库管理系统查询优化的一般步骤。

记忆：画语法树，优化，存取路径，查询计划

```
（1）把查询转换成某种内部表示，通常用的内部表示是语法树
（2）利用优化算法，优化语法树
（3）选择低层的存取路径
（4）生成查询计划并选择其中代价最小的一种
```

11 数据库恢复技术

11.1 事务的基本概念

事务（transaction）：用户定义的一个数据库操作序列，不做或全做，这个工作单位不可分割。

显式定义事务

```
BEGIN TRANSACTION;
COMMIT;       //正常结束，提交，所有更新写回物理数据库
ROLLBACK;     //发生错误，回滚，事务中发生的操作全部撤销
```

★事务的ACID特性

- 原子性(atomicity)：事务中的操作要么不做要么全做。
- 一致性(consistency)：事务执行的结果必须使数据库从一个一致性状态转移到另一个一致性状态。即完整性。

- 隔离性(isolation)：一个事务的执行不能被其它事务干扰。
- 持续性(durability)：一个事务一旦提交，它对数据库中数据的改变是永久性的，接下来其它操作或故障不会对它的执行结果造成影响。

事务ACID遭破坏的可能因素：

- 多个事务并行运行，不同事务的操作交叉执行（并发控制机制）
- 事务在运行过程中被强行停止（恢复机制）

★11.3 故障的种类

- 事务内部的故障：事务没有到达预期的终点（COMMIT或显式ROLLBACK）
- 系统故障：（软故障）造成系统停止运转，系统要重新启动（尚未完成的数据送入物理数据库，破坏原子性/一致性；内存/缓冲区数据丢失，破坏事务持续性）
- 介质故障：（硬故障）外存故障：磁盘损坏，磁头碰撞，瞬间强磁场干扰，破坏硬盘数据的计算机病毒等。

11.4 数据库恢复的实现技术

- 本质：建立冗余数据
- 方法：数据转储（dump）和登记日志文件

分类：

- 事务故障&系统故障：记录事务更新对象旧值和新值（日志文件），回滚未提交的事务（恢复为旧值），重做已提交的事务（更新为新值），（日志logging技术）
- 介质故障：重载备份（数据备份backup技术）

数据转储（数据备份backup技术）

- 静态备份：备份开始时处于一致性状态，且没有事务正在进行（优：实现简单；缺：必须等待事务结束，新事务等待备份结束）
- 动态备份：备份操作与用户事务并发进行（优：不用等待结束，不会影响新事务进行；缺：仅依靠后备副本无法保证数据有效）
- 全量备份
- 增量备份

登记日志文件（日志logging技术）

- Undo日志：先写日志，后写数据，最后写提交标记
- Redo日志：先写日志，再写提交标记，最后写数据
- Undo/Redo日志：先写日志，后写数据

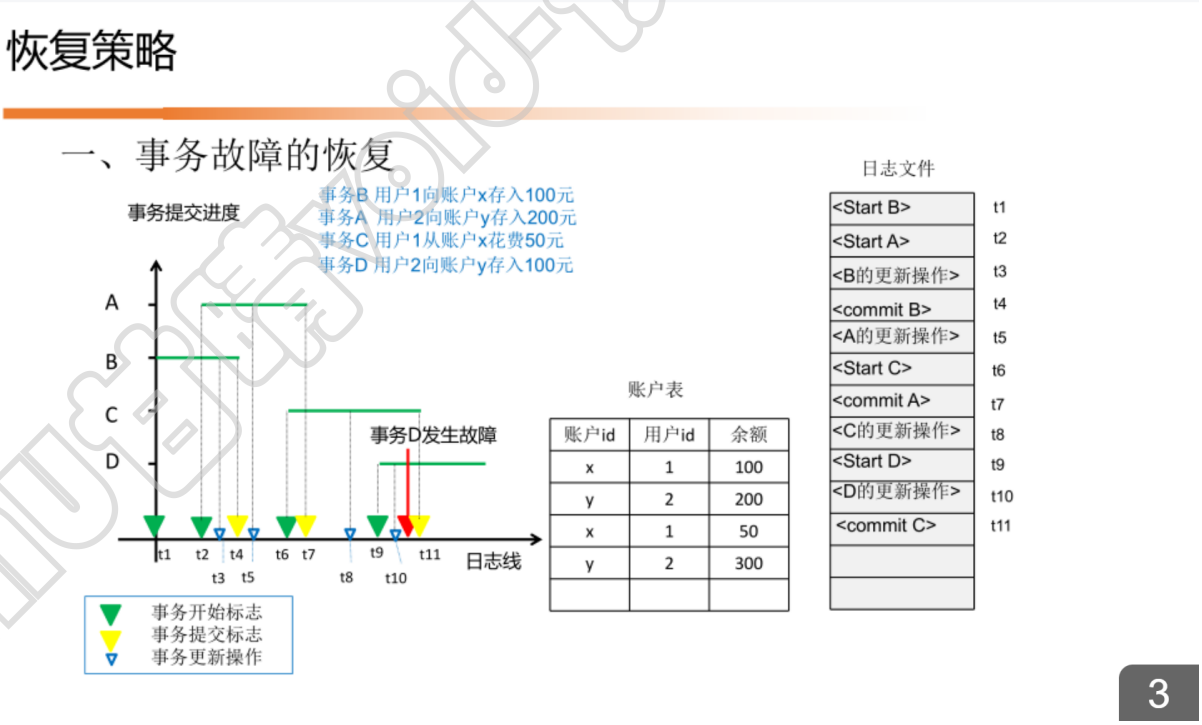
★为什么要先写日志，后写数据？

- 这两者之间可能发生故障，若先写日志后发生故障，恢复时多做一次不必要的Undo操作，不会影响数据库正确性。若先写数据后发生故障，因为没有日志所以无法恢复。

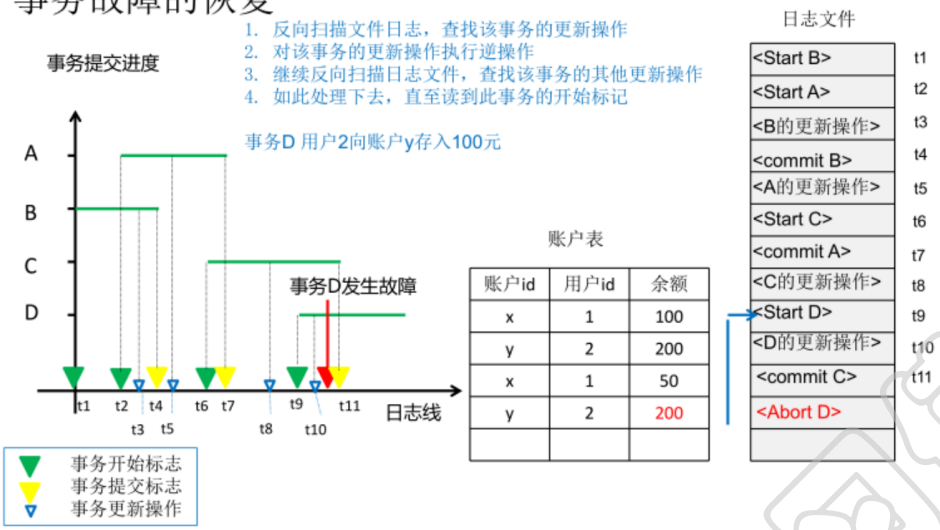
★11.5 恢复策略

事务故障：

- 因为只是该事务发生的故障
- 反向扫描日志文件，对该事务的更新操作做逆操作，直到读到此事务的开始标记



一、事务故障的恢复

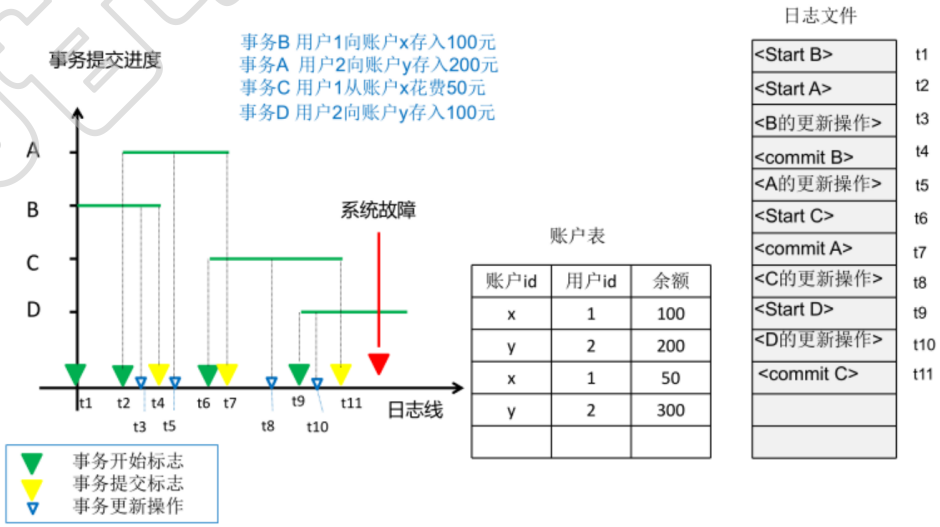


4

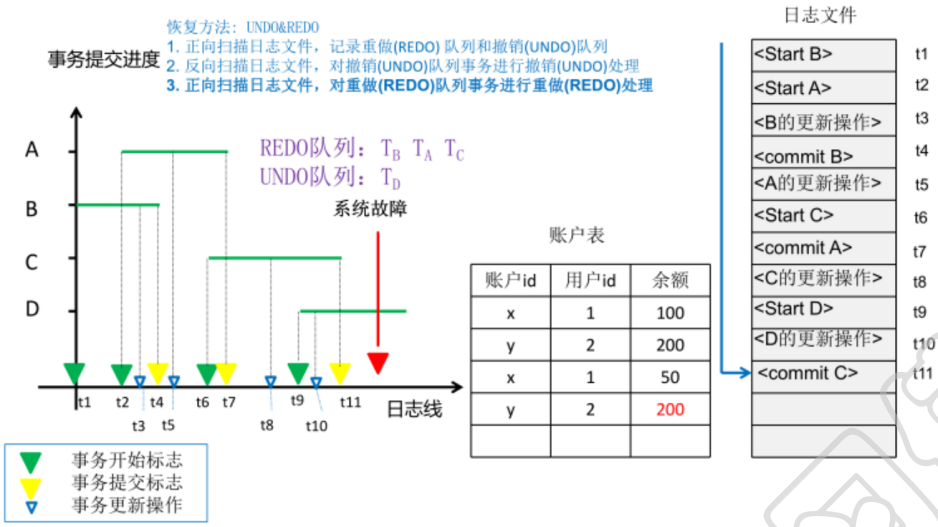
系统故障：

- 正向扫描日志文件，找出故障发送前已提交的事务（有BEGIN TRANSACTION 和COMMIT），放入REDO-LIST。找出故障时尚未完成的事务（只有BEGIN TRANSACTION 但无 COMMIT），放入UNDO-LIST。
- 对UNDO-LIST中的事务做撤销（反向扫描日志文件，对每个更新操作做逆操作，即“更新前的值”写入数据库）
- 对REDO-LIST中的事务做重做（正向扫描日志文件，重新执行日志文件登记操作，即“更新后的值”写入数据库）

二、系统故障的恢复



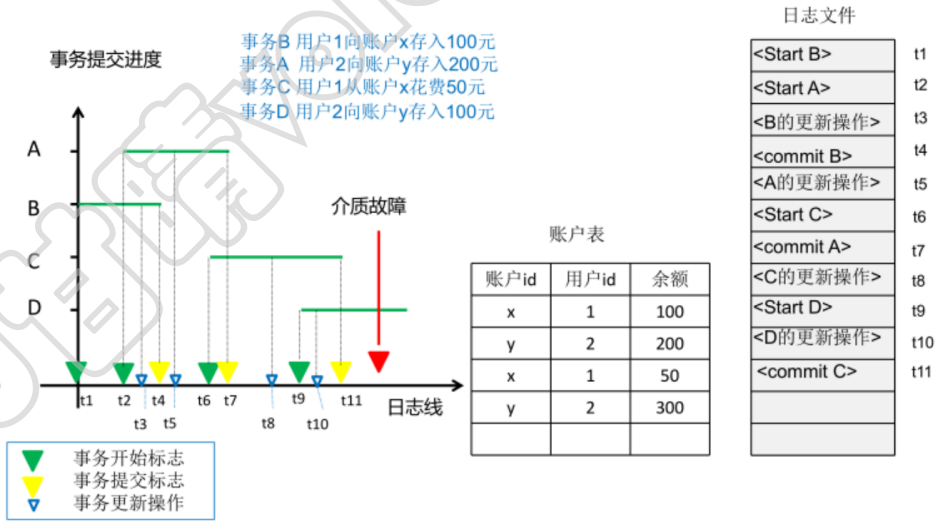
二、系统故障的恢复



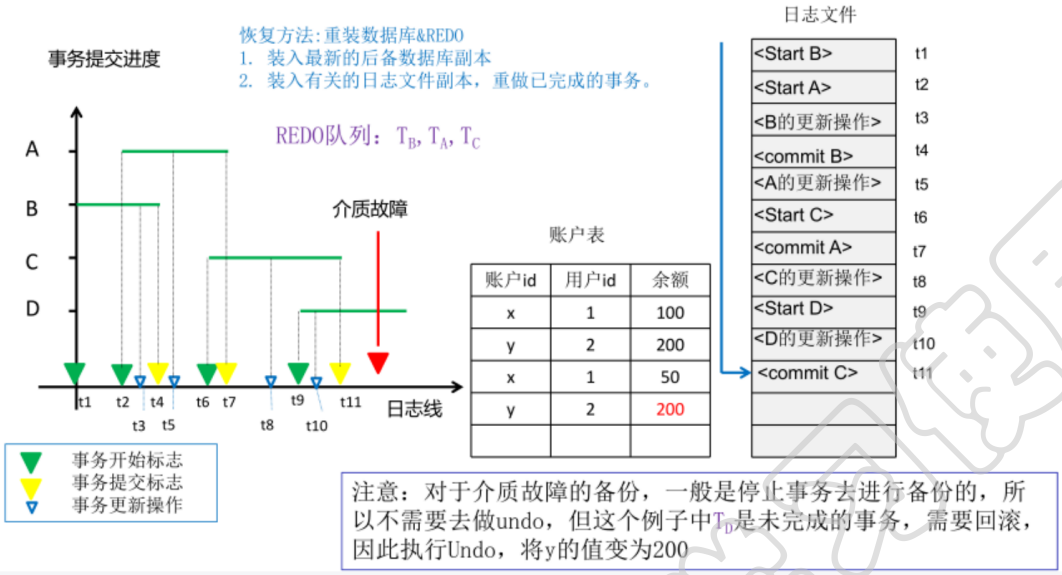
介质故障：

- 装入最新数据库后备副本，恢复数据库到最近的一致状态
- 装入相应的日志文件副本，重做已完成的事务

三、介质故障的恢复



三、介质故障的恢复

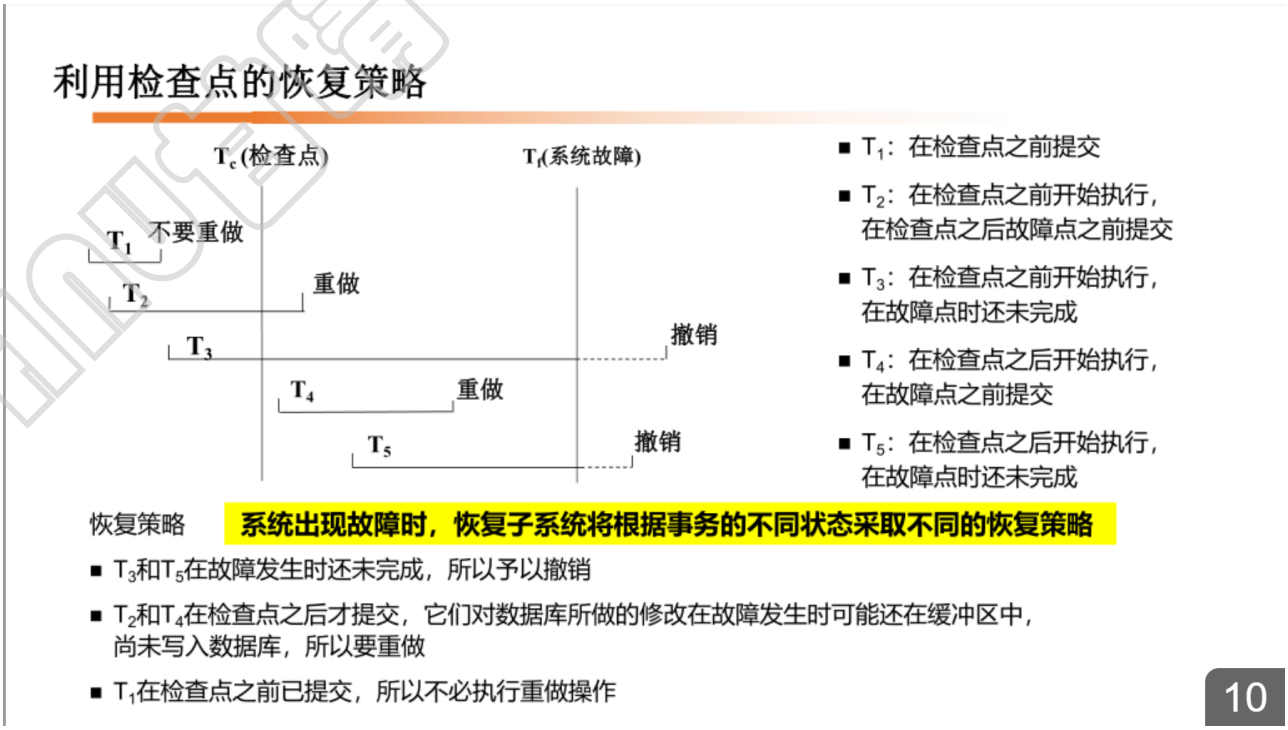


★11.6 具有检查点的恢复技术

问题：搜索整个日志很耗时，重新执行浪费大量时间，日志存储代价很大。为了解决这个问题，新增检查点。

检查点记录{①建立检查点时刻所有正在执行的事务清单 ②这些事务最近一个日志记录的地址}

重新开始文件：存放各个检查点在日志文件中的地址



利用检查点恢复的步骤：

- 从“重新开始文件”中找到最后一个“检查点记录”的地址，在日志文件中找到最后一个“检查点记录”。
- 建立事务队列UNDO-LIST和REDO-LIST，检查点时的ACTIVE-LIST初始放入UNDO-LIST
- 正向扫描直至结束：若某任务START，放入UNDO-LIST；若某任务COMMIT，放入REDO-LIST
- 对UNDO-LIST中的事务，从结束逆向到最后检查点开始做UNDO，但对于在检查点记录中的ACTIVE事务，要继续回推到该事务的START（总结，UNDO-LIST中的事务都要完整从结束反推回START）
- 对REDO-LIST中的事务，从最后检查点开始正向扫描，对每一条做REDO

具体案例：

检查点技术

■ [例] 系统崩溃时日志文件记录内容如图所示，试写出系统重启后恢复处理的步骤及恢复操作(指UNDO, REDO操作)，并指明A, B, C, D恢复后的值分别是多少?

■ 分析过程如下：

	UNDO-list	REDO-list
<Checkpoint {T_1, T_2}>	{T_1, T_2}	{ }
<T_3, START>	{T_1, T_2, T_3}	{ }
<T_1, COMMIT>	{T_2, T_3}	{T_1}
<T_4, START>	{T_2, T_3, T_4}	{T_1}
<T_3, COMMIT>	{T_2, T_4}	{T_1, T_3}

<T_0, START>
<T_1, START>
<T_0, A, 2, 12>
<T_2, START>
<T_0, COMMIT>
<T_1, C, 6, 16>
<T_2, B, 4, 14>
<Checkpoint {T_1, T_2}>
<T_1, A, 12, 20>
<T_3, START>
<T_1, COMMIT>
<T_2, B, 14, 40>
<T_3, A, 20, 60>
<T_4, START>
<T_3, D, 8, 18>
<T_3, COMMIT>
<T_4, C, 16, 30>

图 日志文件

15



11.7 数据库镜像（了解）

复制主数据库到别的地方，可以同时便利并发操作的读取。

12 并发控制

单处理机系统中，事务的并行执行实际上是这些并发事务的并行操作轮流交叉运行。称为：交叉并发方式。

12.1 并发控制概述

事务是并发控制的基本单位。目标是保证事务的隔离性和一致性。

并发操作带来的不一致性主要有：

- 丢失修改（T1和T2读入同一数据各自修改，T2的提交破坏了 T1的修改，导致T1的修改被丢失）
- 脏读（T1提交的数据被T2读取后又被撤销，此时T2读到的数据就是脏的，即不正确的）
- 不可重复读（T1读取数据后，T2对记录中的数据更新，使得T1再次读取时得到与之前不一致的值）
- 幻读（T1读取数据后，T2对部分记录插入或删除，使得T1再次读取时发现一些记录小时了或者多出来了）

并发控制的主要技术：封锁（locking），时间戳（timestamp），乐观方法，多版本并发控制

12.2 事务的隔离级别

由低到高（数据一致性增强，系统代价增高）：

- 读未提交：允许一个事务读取另一个未提交事务正在修改的数据（避免丢失修改）
- 读已提交：只允许一个事务读取一个已提交的数据（避免丢失修改，避免脏读）
- 可重复读：一个十五开始读取数据后，其他事务就不能再对该数据执行更新操作（避免丢失修改，避免脏读，避免不可重复读）
- 可串行化：事务执行顺序可串行化（避免所有）

12.3 封锁

封锁：事务T对某个数据对象操作前先向系统申请加锁

基本封锁类型：

- 排他型锁（X锁）：写锁，只允许事务T读取修改A，不允许别的事务加锁。
- 共享型锁（S锁）：读锁，允许事务T读取A不能修改A，允许别的事务加S锁但不能X锁。

相容矩阵：只有S锁和S锁自己可相容，另：空跟别的都可相容

12.4 封锁协议

三级封锁协议：定义何时申请锁，持续时间，何时释放

- 一级封锁协议：长写锁。事务T修改数据R前加X锁，事务结束后释放。（【没有丢失修改】不保证不脏读，不保证可重复读）
- 二级封锁协议：长写锁+短读锁。在一级协议上，新增：事务T读数据R前加S锁，读完就可释放。（【不脏读】不保证可重复读）
- 三级封锁协议：长写锁+长读锁。在一级协议上，新增：事务T读数据R前加S锁，事务结束才可释放。【可重复读】

12.5 活锁和死锁

活锁

原因：多个事务请求同一个，但有可能先允许了后面的事务，导致前面的事务一直等待，饿死。

解决方法：采取先来先服务的策略

死锁

原因：T1和T2各自封锁对方想要获得的资源，并同时请求自己想要获得的资源

死锁的预防；

- 一次封锁法：一次性将所有数据全部加锁
- 顺序封锁法：预先规定封锁顺序，所有事务都按照这个顺序封锁。

死锁的诊断与解除：

- 超时法：超过规定时间就认为发生了死锁
- 事务等待图法：有向图，节点为事务，边指向等待的事务，若出现回路，则存在死锁

12.6 并发调度的可串行性

调度不同，可能结果不同，存在一种正确调度。

- 串行调度：（串行调度的并发执行必然是正确的）
- 可串行化调度：这种调度方式产生的结果与某次串行调度产生的结果相同。（可串行化调度的并发执行是正确的）
- 冲突可串行化调度：这种调度在保证冲突操作次序不变的情况下，通过交换两个事务不冲突操作的次序得到的另一个调度是串行的，则该事务是冲突可串行化的，也必然是可串行化的（充分条件：冲突可串行化→可串行化）
- 冲突操作：不同事务对同一数据的读写/写写操作
- 【不同事务的冲突操作】和【同一事务的两个操作】顺序不能互换

做题：判断一个调度是否冲突可串行化调度？在规则允许的范围内尽可能交换事务中各语句的次序，使得事务能尽可能分离开来，变成串行的，如果能办到，就是冲突可串行化调度。

【期末考试8分】例题：R3(B)R1(A)W3(B)R2(B)R2(A)W2(B)R1(B)W1(A)，该调度是冲突可串行化的调度吗？为什么？

对原式中R1(A)依次与右边的邻居交换最终得到
R3(B)W3(B)R2(B)R2(A)W2(B)R1(A)R1(B)W1(A)这是一个串行调度，故原调度是冲突可串行化调度。

12.7 两段锁协议

为了保证调度可串行化，使用两段封锁（two-phase lock，2PL）来实现可串行化。

遵守两段锁协议的一定是可串行化的。

所有事务必须分两个阶段对数据项加锁和解锁（读写之前申请封锁，释放锁后，不再申请和获得封锁）

- 扩展阶段：获得封锁
- 收缩阶段：释放封锁

★一次封锁法<>两段锁协议

一次封锁法遵守两段锁协议，但两段锁协议不要求一次封锁，故两段锁协议仍然可能发生死锁

12.8 封锁的粒度

封锁粒度：封锁对象的大小（可以是逻辑单元：属性值，属性值集合，元组，关系，索引项，整个索引，整个数据库；物理单元：数据页，索引页；物理记录等）

封锁粒度↑，并发度↓，系统开销↓

多粒度封锁

多粒度树（根节点表示整个数据库，往下层次依次关系，元组等）

多粒度封锁协议：允许该树中的每个节点被独立加锁，加一个节点的锁时，其子树上的节点都加锁

- 显式封锁：直接加到该节点的锁
- 隐式封锁：由于其上级节点被加锁而被加锁

意向锁：表明该节点的子树上的节点正在被加锁。对任意节点加锁时，其所有上层节点都要加意向锁。

- 意向共享型锁（IS）：其子树上的节点想加S锁

- 意向排他型锁（IX）：其子树上的节点想加X锁
- 共享意向排他型锁（SIX锁）：先加S锁再加IX锁
- SIX例子：对某个表加SIX锁，读取整个表，但更新部分元组。
- 锁的强度关系：X>SIX>S,IX>IS

具有意向锁的多粒度封锁方法，提高了系统并发度，减少了加锁和解锁开销。

Void学习笔记

【期末考试复习】

【老师提供】数据库系统期末考试复习提纲（2023）

第一章 绪论

1.1 数据库的 4 个基本概念（数据、数据库、数据库管理系统和数据库系统）

- 数据，是描述事物的符号记录。
- 数据库（DB），是长期存储在计算机内、有组织、可共享的大量数据的集合。
- 数据库管理系统（DBMS），位于用户与OS之间的数据管理软件。
- 数据库系统（DBS），是由DB、DBMS及其应用开发工具、应用程序和DBA组成的存储、管理、处理和维护数据的系统。

1.2 数据库系统的特点、数据库系统与文件系统的区别与联系

- 特点：数据共享性高，冗余度低且易扩充，独立性高，整体数据的结构化。
- 联系：文件系统和数据库系统都用于存储和管理数据

1.3 数据模型及三要素

- 数据模型：对现实世界数据特征的抽象
- 三要素：数据结构，数据操作，完整性约束
- 数据结构：描述数据库的组成对象以及对象之间的联系。
- 数据操作：对数据库中各种对象的实例允许执行的操作的集合，包括操作及有关的操作规则。
- 完整性约束：一组完整性规则。

1.4 数据库系统的三级模式结构（外模式、模式、内模式）

数据库的二级映像功能与数据独立性（逻辑独立性、物理独立性）

第二章 关系数据库

2.1 码、主属性、非主属性、外码

2.2 关系代数 并、差、交、笛卡尔积、选择、投影、连接、除运算。

2.3 关系代数查询及关系运算

第三章 关系数据库标准语言 SQL

3.1 SQL 的基本概念，基本表、视图。

3.2 基本表及视图的定义、查询、更新

3.3 数据查询: 常用的 SQL 语句，常用的查询、修改、删除、分组、排序等 SQL 语句

3.4 能熟练使用聚集函数、通配符

3.5 多表连接查询、复合条件连接查询、嵌套查询

3.6 数据更新 插入数据、修改数据、删除数据

第四章 数据库安全性

4.1 数据库的安全性

- 保护数据库，以防不合法使用所造成的数据泄露，篡改或破坏

4.2 实现数据库安全性控制常用的方法和技术

4.3 掌握用户、角色的创建及授权和收回权限

第五章 数据库的完整性

5.1 数据库的完整性

- 数据库的正确性和相容性

5.2 实体完整性、参照完整性、用户定义完整性

第六章 关系数据理论

6.1 对函数依赖、码、主属性、非主属性、外码等有深入的理解

6.2 掌握关系模式的基本函数依赖、关系模式的候选码

6.2 掌握规范化理论，能够判断给定关系模式的规范化程度

6.3 掌握模式的分解,能够按要求对给定关系模式进行分解，修改为符合更高一级的范式

第七章 数据库设计

7.1 掌握数据库的一般设计过程及每个阶段的主要任务。

- 需求分析：明确用户对数据库的需求
- 概念结构设计：设计独立于具体DBMS的概念模型
- 逻辑结构设计：根据概念模型转换得到具体的DBMS逻辑模型
- 物理结构设计：设计存储结构和存储路径
- 数据库实施：
- 数据库运行和维护：

7.2 根据语义设计系统的 E-R 图，并将 E-R 图转换为关系模式（写出关系名称和所有属性）

7.3 对数据库设计方面有自己独到的理解

- 【猜测】可扩展性：数据库模块化，数据库模块可变更的能力

第九章 关系查询处理和查询优化

9.1 理解关系数据库系统查询优化的主要规则及步骤

- 记忆（主要规则）：选择怎么样？投影怎么样？选择和投影可以怎么样？选择和笛卡尔积可以怎么样？公共子表达式？连接算法？
- 记忆（步骤）：画语法树，优化，存取路径，查询计划

9.2 了解代数优化、物理优化

- 代数优化：对关系代数表达式的优化，按照一定规则对关系代数表达式进行等价变换，改变操作的组合及次序，提高查询效率
- 物理优化：对存取路径和底层操作算进行优化

第十章 数据库恢复技术

10.1 对事务的基本概念及事务的 ACID 特性有一定的理解和掌握

【事务】

10.2 理解数据库系统中故障的种类

- 事务，介质，系统

10.3 掌握数据库恢复技术（数据转储或数据备份、登记日志文件）

10.4 掌握并能熟练使用具有检查点的恢复技术。

- 【推测考大题】

第十一章 并发控制

11.1 并发控制及并发操作带来的数据不一致性

- 丢失修改，脏读，不可重复读

11.2 封锁技术及基本封锁类型（排他锁和共享锁）、三级封锁协议

11.3 活锁和死锁，死锁的预防、诊断与解除

11.4 可串行化调度及两段锁协议，判断一个并发调度是不是冲突可串行化调度。

【两段锁协议】

11.5 了解封锁粒度及多粒度封锁。

HDU电脑void小组学习组